

# DCLT Portal — Conceptual Architecture

Last updated: May 1, 2026. Supersedes the previous version and absorbs the prior TA Update Brief.

## Purpose

This document is the conceptual reference for the DCLT Portal: data model, disciplines, vocabulary, implementation constraints, migration sequence.

Claude Code maintains this document alongside the codebase. The code is the technical architecture — there is no separate technical architecture document. When implementation surfaces a model gap, this document is updated first; the code follows.

## Scope

This document defines:

- The Tag / Layer dichotomy and the authorship test that separates them
- Layer source types (External / Derived / Dynamic) and their refresh disciplines
- Tag dimension structure (state space, default, applicability, transitions)
- Composition rules between Tags and Layers, and cycle prevention
- Filtering semantics: curation, idioms, combination, cross-node, applicability
- Node types, including links as relationships between nodes
- Naming canon
- Worked examples grounding the abstractions
- Constraints on implementation
- Migration sequence

This document does not specify schema DDL, file layouts, ingest implementation, materialization mechanics, evaluator design, query patterns, or UI surface implementation. Those are decisions for Claude Code against the actual codebase, governed by the constraints below.

# Guiding principles

Eight principles drive the model. The first four are commitments — what the architecture exists to support. The last four are disciplines — how the architecture stays coherent as it grows.

**Hard data is gettable.** What's been ingested was hard-won; what's asked for next is probably feasible. The architecture encourages incremental sourcing rather than presuming everything is known up front.

**The interface serves the question.** Same data, different lenses, depending on what's being decided. Filtering and presentation are first-class concerns, not an afterthought on top of storage.

**Human judgment is captured, not lost.** Tags, notes, and determinations from people who know the ground are first-class data, distinct from machine guesses, with full provenance.

**Lineage is non-negotiable.** Every answer traces to its source. The split between machine-extracted and human-confirmed is preserved in every read.

**Authorship is the seam.** Every design question routes back to: who authored this value? That's what determines Tag vs Attribute, what makes the append-only discipline non-negotiable, why "system tag" is banned vocabulary. Contested calls have a clear test.

**Re-ingest must not disturb human work.** The most important operational invariant. Drives the Tag/Layer split, the append-only log, applicability-as-projection (so attribute changes don't write tag rows), and the reference.db / transactions.db separation. If a proposed change can disturb human work on re-ingest, the proposal is wrong.

**One way to express each shape.** When two patterns could express the same thing, pick one. Cross-node tag filters are existential; counts are Dynamic Layers. Applicability is a query projection, not a state. Dimensions live in code; filter curation lives in configuration. Picking one keeps the model legible as it grows.

**Complexity follows demand.** The model handles the cases in front of us cleanly and leaves room for the cases that haven't shown up. Edge cases get positions when a real request surfaces them, not when they're imagined. A simple rule that covers the current need beats an elaborate rule that anticipates needs that may never arrive. When extension is required, the resolution path is the one already defined: name the gap, take a position in this document, propagate to code.

## The loop

The architecture exists to support a four-step cycle:

1. **Surface what's there.** External Layers source from feeds; Derived Layers calculate from those.
2. **Ask what we could do with it if accurate.** Dynamic Layers and queries pose the question.
3. **Let people who know the ground convert guesses into determinations.** Tag dimensions are the workflow.
4. **Re-ask with stronger footing.** Each pass, the latest fold answers with more credibility.

Re-ingest never disturbs human work — it lives in Tags. Changing a Layer formula never disturbs history — Layers don't have history. Changing a calculation never disturbs storage — Derived Layers re-materialize on next ingest, Dynamic Layers re-evaluate on next read, neither touches `transactions.db`.

## Where this lives today

Single-server portal, VPS-hosted, GitHub-deployed. A local processing pipeline produces a read-only `reference.db` containing External and Derived Layers, deployed alongside the application. `transactions.db` lives on the server, append-only, never overwritten by deployments. Three-pane parcel view is the primary UI surface; document-centric views and cover-page hygiene rollups are near-term.

## The dividing line

Authorship in the portal is the only test.

**Attribute.** Not authored by users in the portal. Sourced from external feeds, derived by calculation from other Layers, or computed dynamically using Tag inputs. Lives in a Layer.

**Tag.** Authored by users in the portal. Default-seeded so the dimension is queryable from day one, then assorted by users as work progresses. Lives in `transactions.db`.

The seam is authorship of the current value, not authorship of the seed. A Tag whose default is a system suggestion is still a Tag — the seed is a proposal, the user's assortment is the verdict, and human authorship beats the seed regardless of timing. An Attribute whose calculation depends on user-authored Tags is still an Attribute — no user assorted the calculated value directly.

## Layers (attributes)

Every Attribute belongs to a Layer. A Layer has two classifying axes.

## Source type

Three values.

**External.** Sourced from a feed. Assessor exports, MassGIS layers, Registry OCR output, Dave’s ownership spreadsheet.

**Derived.** Calculated from other Layers (External or Derived only). Materialized in `reference.db` . Refreshed on ingest.

**Dynamic.** Calculated using at least one Tag fold as input. Never materialized. Evaluated at read time.

A Layer is Dynamic if it reads any Tag fold, Derived otherwise. Classification follows the inputs, not the intent. Adding a Tag input to a Derived Layer reclassifies it — code review catches the change.

## Node type

The output key. Parcel, Document, ParcelDocumentLink, or any future node type. Future node types are added by code change, not configuration.

A Layer that aggregates document data into per-parcel rollups is a parcel-typed Layer with document inputs — node type follows the output, not the inputs.

Links are a node type. A link’s `target_id` is a composite of the related nodes’ identifiers (e.g., `parcel_id + document_id` ). Suspected links are produced by machine processes; user adjudications on those suspicions are Tags on the link node. Treating links as a node type keeps the model uniform — link Tags and link Attributes follow the same rules as everything else.

## Naming

`<NodeType><Subject>` for derived and dynamic Layers, e.g., `ParcelFootprintRatio` , `DocumentKeywordScores` , `ParcelCoverageRollup` , `SuspectedParcelDocumentLink` . External Layers may inherit names from their source schema.

## Storage and refresh

| Source type | Storage                                  | Refresh |
|-------------|------------------------------------------|---------|
| External    | <code>reference.db</code>                | Ingest  |
| Derived     | <code>reference.db</code> (materialized) | Ingest  |
|             |                                          |         |

|         |                          |          |
|---------|--------------------------|----------|
| Dynamic | In memory (formula only) | Per read |
|---------|--------------------------|----------|

If a Dynamic Layer becomes a performance problem, the escalation is an overnight materialization job — caching, not a storage commitment. The Layer's classification doesn't change; only its read path does.

## Tags

A Tag is a dimension for assorting nodes (parcels, documents, links, others) with a finite state space. Users move records through that space. The current state on a dimension is the latest-fold of the tag log for that `(target_type, target_id, dimension)`.

## Mechanics

Append-only log in `transactions.db`. Every event records target `(target_type, target_id)`, dimension, state, `user_id`, server-assigned monotonic `seq`, client `created_at`, optional note. No UPDATE, no DELETE — corrections append a new event. Latest row wins per `(target_type, target_id, dimension)`. Full history preserved and queryable.

A database-level trigger rejects UPDATE and DELETE on the tag table. Application code paths must never construct such statements.

## Dimension definition

Each Tag dimension is defined by six properties, all in code:

1. **Name.** `CoverageDetermination, IdentityResolution, Article97Determination, LinkAdjudication, DCLTOwnership`.
2. **Node type.** Which target type the dimension applies to.
3. **State space.** Closed set of valid states. Includes `Unconfirmed` if the dimension is a decision-pending workflow. Does not include `Not Applicable` — applicability is a separate axis (see Filtering).
4. **Default rule.** How new records seed. Constant (usually `Unconfirmed`), or a function of attributes. Default rules read External or Derived Layers only — never Dynamic — to keep the dependency graph acyclic.
5. **Applicability rule.** Which records the dimension is meaningful for. May depend on attributes. `IdentityResolution` is dormant when `IdentityState = OK`. `Article97Determination` only applies to documents with a keyword hit above threshold. `LinkAdjudication` only applies to suspected links.

6. **Allowed transitions.** Usually any-to-any. Sometimes constrained by an attribute:

`IdentityResolution` permits `{ADB Add, GIS Remove}` when `IdentityState = GIS-only`, `{GIS Add, ADB Remove}` when `ADB-only`.

Adding a Tag dimension is a small commit, not a runtime config change. Dimensions are versioned with the code; the log preserves history through definition changes.

Whether a dimension appears in filter chrome and where it groups in detail panels is a separate concern, handled by configuration (see Filtering).

## Hygiene journey

`Unconfirmed` is the work queue. Filter to it, process, watch coverage Layers climb. Derived or Dynamic Layers like “% adjudicated” and “% resolved” make progress visible across the inventory.

## Naming convention

- `-Determination` for categorical verdicts where users decide what an observation means.
- `-Resolution` for dispositions on a discrepancy.
- `-Adjudication` for verdicts on machine-suspected relationships (links).
- Plain name where the dimension itself is a workflow with no parallel attribute (`DCLTOwnership` when the source is users-marking-in-the-portal).

One suffix per Tag shape keeps the model legible as the dimension count grows.

## Composition

Layers and Tags compose by reading from each other under constrained rules.

**Layers can read Tags.** A Layer that reads any Tag fold is Dynamic. This is the only way a Tag’s current state influences a calculated value.

**Tag default and applicability rules can read Attributes.** Default seeds and applicability conditions may depend on External or Derived Layers. They may not depend on Dynamic Layers — that would create a cycle (Tag → Dynamic Layer → Tag).

**Tags do not read Tags.** A Tag’s state space and seed are independent of other Tags’ folds. If a workflow seems to require otherwise, the right shape is a Dynamic Layer that combines the Tags, queried wherever the combination matters.

**Attributes and Tags never write to the same row.** They live in different stores with

different keys. The “human beats system” fold rule is automatic, not enforced — there is nothing for system writes to overwrite.

## Filtering

Filtering is a first-class concern. Three axes are visible to the user: tag fold state, attribute value, and applicability. The first two are filter selections; the third is an automatic projection from the dimension’s applicability rule.

### Filter curation

Each node type carries more dimensions than belong in any one filter chrome. Which Tags and Attributes appear in the filter UI for a given node type is configuration, not code. The same goes for grouping in detail panels. Filter curation is editable without redeploy and lives in `transactions.db` alongside other portal state.

The split: code defines what a dimension means (the six properties for Tags, the formula and source for Attributes); configuration decides where it appears in the UI.

### Tag filters

When a user filters on a Tag, the UI offers a multi-picker over the dimension’s state space. A record matches if its current fold is among the selected states and the dimension is applicable to it. If the dimension’s applicability rule is conditional, the picker also offers `Not Applicable` as a structurally separate affordance — visually distinct from state choices, since it filters on a different axis.

For dimensions whose applicability rule is “all records,” `Not Applicable` is omitted (the result would always be empty). For dimensions like `IdentityResolution`, `Article97Determination`, or `LinkAdjudication`, `Not Applicable` is present and meaningful.

Within a single Tag filter, state values are unioned (OR). Selecting both states and `Not Applicable` returns the union.

### Attribute filters

Filter idiom follows attribute shape:

- **Limited-set values.** Multi-select picker.
- **Numeric continuum.** Range slider.
- **Boolean.** Degenerate two-option multi-select.

- **Free-form text.** Not filterable; appears in detail panels only.

Attributes can have applicability rules just as Tags do. A coverage ratio is only meaningful for parcels with known footprint and acreage. The `Not Applicable` affordance applies the same way.

## Combining filters

Selected filters across dimensions intersect (AND). A record matches if it satisfies every selected filter clause. Within a single filter (a tag's state multi-select, an attribute's range), values are unioned (OR).

Order of selection doesn't matter. The filter set is a conjunction; the user assembles clauses, the system intersects.

## Cross-node filters

A filter on one node type can include clauses on dimensions of a different node type, joined by a relationship. The first case the model addresses: parcel filters with document-scoped clauses, joined by parcel-document links.

Semantics are existential: a parcel matches if any linked document satisfies the document-scoped clause. Counts and thresholds belong in Dynamic Layers, exposed as parcel attributes — not in the cross-node filter itself.

The link adjudication is part of the join. By default, only links with `LinkAdjudication = Confirmed` participate. Unadjudicated links are noise; rejected links are wrong. The default makes the filter a forcing function for adjudication work — a user who needs a parcel to satisfy a document-scoped clause must first confirm the relevant link. Engagement is the intent.

When the machine signal is strong enough that adjudication isn't yet feasible, the related Derived Layer (e.g., a suspected-link confidence score) is available as an attribute filter on the link. That's the documented escape hatch from the engagement gate.

## Rollup denominators

Derived and Dynamic Layers that report dimension-level percentages count among the applicable population by default. `ParcelArticle97Rollup`'s "% confirmed" reads against documents with keyword hits, not against all documents — otherwise the hygiene number is structurally diluted. The `Not Applicable` count is reported separately when relevant, never folded into the active denominator.

## Applicability flips



Applicability is computed at read time from current attribute values. A record can transition between applicable and not-applicable as upstream data changes. When that happens, the existing Tag fold is preserved in history and remains queryable, but the record falls under `Not Applicable` for active filters until applicability is restored.

Example: a parcel was `GIS-only`, the user resolved it as `ADB Add`, and the next ingest updates the Assessor's database so `IdentityState` flips to `OK`. The fold remains `ADB Add` in history; the active filter shows the parcel as `Not Applicable`. The resolution did its job and the dimension stepped back.

## Examples

### Coverage

`ParcelFootprintRatio` is a Derived Layer (footprint area ÷ parcel area, computed from External Layers).

`CoverageDetermination` is a Tag.

- **States:** {Unconfirmed, Undeveloped, Underdeveloped, Developed}
- **Node type:** Parcel
- **Default:** Unconfirmed
- **Applicability:** all parcels
- **Transitions:** any-to-any

`ParcelCoverageRollup` is a Dynamic Layer producing inventory-level percentages by determination state — drives the cover-page hygiene journey.

### Identity

`IdentityState` is a Derived Layer with states {OK, ADB-only, GIS-only}, calculated each ingest from External Layers (Assessor and GIS presence).

`IdentityResolution` is a Tag.

- **States:** {Unconfirmed, ADB Add, ADB Remove, GIS Add, GIS Remove}
- **Node type:** Parcel
- **Default:** Unconfirmed
- **Applicability:** only when `IdentityState` ≠ OK

- **Transitions: constrained by** `IdentityState` ( `GIS-only` → {`ADB Add`, `GIS Remove`} ; `ADB-only` → {`GIS Add`, `ADB Remove`} )

## Article 97

`DocumentArticle97KeywordScore` is an External Layer (OCR keyword scorer output).

`Article97Determination` is a Tag.

- **States:** {`Unconfirmed`, `Confirmed`, `Denied`}
- **Node type:** Document
- **Default:** `Unconfirmed` when keyword score is above threshold; dimension inapplicable otherwise
- **Applicability:** documents with keyword score above threshold
- **Transitions:** any-to-any among the three states

`ParcelArticle97Rollup` is a Dynamic Layer aggregating document determinations to the parcel level (count confirmed, count unconfirmed, etc.). Denominator is applicable documents.

## Parcel-document link

The link is a node type; suspected links are an Attribute; user adjudications are a Tag.

`SuspectedParcelDocumentLink` is a Layer (External or Derived depending on source). Inputs vary by document source: registry deeds carry book/page references that can be joined against the Assessor's database; town meeting documents require candidate generation that scans full text against parcel identifiers. The Layer is tuned for high recall — false negatives are worse than false positives, because a user can dismiss a wrong suggestion but cannot see a missed one. Per-link attributes record the evidence basis.

`LinkAdjudication` is a Tag.

- **States:** {`Unconfirmed`, `Confirmed`, `Rejected`}
- **Node type:** `ParcelDocumentLink` ( `target_id` is a composite of `parcel_id` and `document_id` )
- **Default:** `Unconfirmed` for every suspected link
- **Applicability:** only suspected links — there is no point adjudicating a relationship no machine ever proposed

- Transitions: any-to-any

Cross-node filters between parcels and documents read this dimension via the confirmed-only default described under Filtering.

## DCLT ownership

Two cases, depending on the source of truth.

If users mark ownership directly in the portal: `DCLTOwnership` is a Tag with states `{Unconfirmed, Owned, Not owned}`, applicability all parcels.

If Dave's spreadsheet is the source of truth and arrives by feed: `ParcelDCLTOwnership` is an External Layer with states `{owned, not listed}` and an as-of date. A Tag may still exist on top to record portal-side overrides when a user notices the spreadsheet is wrong — same shape as Article 97 (attribute observes, tag overrides).

## Migration sequence

The current implementation encodes an older taxonomy: `tag_type='system' / 'user'`, system tags materialized as rows in `taggings`, `dclt.db` as the store name, no Dynamic Layer evaluator, no filter configuration system, no link node type, no attribute filter mechanics, no cross-node query patterns.

Nine ordered steps bring the codebase into alignment. Each step assumes the previous is complete. Steps 1–3 are unblocking — they remove the system-tag conflation. Steps 4–6 are the core refactor. Steps 7–9 are the user-facing surface.

1. **Rename `dclt.db` to `transactions.db`**. Update access functions, config keys, deploy scripts, file paths. Mostly mechanical, but touches operational surfaces.
2. **Establish the dimension registry**. Define the four initial Tag dimensions (`CoverageDetermination`, `IdentityResolution`, `Article97Determination`, `LinkAdjudication`) with full six-property definitions. Single source of truth — no parallel definitions in the database, the UI, or migration scripts.
3. **Move OCR keyword scores and GIS layer presence out of `taggings` into Layer reads**. Data already exists in `reference.db` — read-path change plus row deletion. Retire the `migrate_*_to_tags` scripts. The “system tag” category disappears.
4. **Add applicability and transition validation to the Tag write path**. Drop the `tag_type='system'` check. Validation order: dimension exists and not deprecated, `target_id` well-formed for `target_type`, dimension applicable to target, state in state space, transition allowed.

5. **Introduce the link node type.** Migrate `parcel_link_adjudications` data into `taggings` under `target_type = ParcelDocumentLink`. Establish `SuspectedParcelDocumentLink` as a named Layer in `reference.db`.
6. **Build the Dynamic Layer evaluator.** Where formulas live, how they're invoked at read time, what caching strategy applies, escalation path to overnight materialization. First Dynamic Layers to implement: `ParcelCoverageRollup`, `ParcelArticle97Rollup`.
7. **Build the filter configuration system.** Schema in `transactions.db`. Curatable: which Tags and Attributes appear in each node type's filter chrome, which appear in detail panels, detail-panel grouping. Migrate any hard-coded UI curation into configuration.
8. **Update query patterns.** Tag-fold filters with applicability projection (two-axis pattern); attribute filters by idiom (multi-select, range, boolean); intersection across dimensions (AND); cross-node existential with confirmed-link-only default; documented escape hatch (attribute filter on the link).
9. **Update UI surface.** Two-axis Tag filter (multi-select picker plus structurally separate `Not Applicable` affordance), attribute filter idioms by shape, cross-node filter chrome, filter curation rendering.

The `reference.db` / `transactions.db` split survives unchanged. What changes is what goes on each side of the line.

## Open questions for resolution against the codebase

1. **Module layout for the dimension registry.** Single file vs. directory; declarative dict vs. classes vs. dataclasses. Pick what fits the existing code style.
2. **Composite `target_id` encoding for `ParcelDocumentLink`.** Only requirement: `target_id` uniquely identifies the link. Recommend a stable, parseable encoding.
3. **Caching strategy for Dynamic Layers.** Request-scoped, time-windowed, or no cache initially. Caching is permitted; storage commitment is not.
4. **Filter configuration editing surface.** Separate admin route or embedded in existing admin UI.
5. **Migration data integrity for existing system tagging rows.** Drop after Layer reads work, or archive in a side table for rollback.
6. **Naming alignment for existing Layers.** Whether to fully rename to the convention (`ParcelFootprintRatio`, `DocumentKeywordScores`) or keep current names with the convention as aliases. Lean toward full rename — the canon is enforced — but call out cost if it's high.

# Constraints on implementation

The code must not:

- Redefine, rename, or split the categories defined here (Tag, Layer, Attribute, Dimension)
- Introduce parallel vocabulary or new top-level abstractions without first proposing them as a CA change
- Decide composition rules that contradict the rules in this document
- Add states like `Not Applicable` to a dimension's state space
- Materialize Dynamic Layers as a default storage strategy (caching is acceptable; storage commitment is not)
- Allow UPDATE or DELETE on Tags under any circumstance
- Embed filter curation in code

When implementation surfaces a gap or contradiction, the resolution path is to propose an update to this document, get it accepted, then implement against the updated model. The code does not extend the model unilaterally.

## Naming canon

These terms are used verbatim. Synonyms and parallel vocabulary fragment the model.

**Categories.** Tag, Layer, Attribute, Dimension.

**Layer source types.** External, Derived, Dynamic.

**Node types.** Parcel, Document, ParcelDocumentLink. Future node types added by code change, not configuration.

**Tag mechanics.** fold (verb and noun), latest-wins, append-only, target ( `target_type` , `target_id` ), seq , applicability, applicable, `Not Applicable` .

**Dimension parts.** state space, default rule, applicability rule, allowed transitions, transition.

**Tag suffixes.** -Determination , -Resolution , -Adjudication , plain workflow name.

**Filtering.** filter curation, filter chrome, detail panel, multi-select, range slider, cross-node filter, existential, confirmed-link-only.

**Stores.** `reference.db` , `transactions.db` .

**Pipeline.** ingest, materialize, evaluate, refresh.

## Banned vocabulary:

- “system tag,” “user tag,” “automatic tag” — the old conflation. Use Tag (always user-authored) or Attribute (anything system-sourced or calculated).
- “calculated attribute” as a stored on-disk artifact other than a materialized Derived Layer. If it reads Tags, it’s Dynamic and lives only as a formula.
- “update tag,” “delete tag” — Tags are append-only. Corrections are new events.
- `Not Applicable` as a state in any state space. It’s a query-time projection.
- `dclt.db` — superseded by `transactions.db`.

## Evolution

This document is canonical for the model but not frozen. Implementation will surface things the model didn’t anticipate. When that happens:

1. The gap or contradiction is named explicitly.
2. A position is taken in this document — new section, amended rule, additional example.
3. The code is updated to match.

This document supersedes prior conceptual architecture documents and the prior TA Update Brief. Future revisions are tracked in version control alongside the codebase.